

Automatic Irrigation System Using Rain Sensor and Soil Moisture Sensor

Project Documentation

Nobel Software Solutions

1. INTRODUCTION

Agriculture plays a vital role in the economic development of many countries, especially in regions where farming is the primary source of livelihood. One of the major challenges faced by farmers is effective water management. Traditional irrigation methods often lead to overwatering or underwatering, which negatively impacts crop yield and wastes valuable water resources. With increasing water scarcity and unpredictable rainfall patterns, there is a growing need for intelligent irrigation systems that can operate automatically based on environmental conditions.

The Automatic Irrigation System Using Rain Sensor and Soil Moisture Sensor is developed to address these challenges by implementing smart sensing and automated control mechanisms. The system monitors soil moisture content using a soil moisture sensor that measures the volumetric water content in the soil. When the soil becomes dry and moisture falls below a certain threshold level, the microcontroller activates the water pump to irrigate the field. Once the desired moisture level is restored, the pump is automatically switched off.

In addition to soil monitoring, the system incorporates a rain sensor that detects rainfall. If rain is detected, the irrigation process is temporarily suspended to prevent water wastage. This ensures efficient utilization of natural resources and reduces dependency on manual supervision.

The integration of sensors with a microcontroller-based control system makes the irrigation process smart and reliable. The automation reduces labour costs and ensures timely watering, which is crucial for healthy crop growth. The system can be implemented in agricultural fields, gardens, and greenhouses. It is affordable, easy to install, and requires minimal maintenance.

By adopting such automated irrigation techniques, farmers can improve crop productivity while conserving water and energy. This project represents a step toward precision agriculture and sustainable farming practices, contributing to better resource management and increased agricultural efficiency.

1.1 Organization Profile

NOBEL SOFTWARE SOLUTIONS is a professional website designing, customized software development, and Business Process Outsourcing – IT/ITES & Internet marketing company providing full-featured web services, including B2B acquisition and B2C e-commerce solutions, and acting as an offshore development centre for overseas development firms. NOBEL SOFTWARE SOLUTIONS is an innovative company, based in India, that provides a series of web-based software applications that have helped its customers create successful business ventures through online initiatives. NOBEL SOFTWARE SOLUTIONS provides all the services a company needs to get online, from web designing to web hosting, and manages leading-edge websites and e-business software solutions for

success in the domestic market. Ever since its inception, Nobel Software Solutions has accrued continuous growth in all its business functions, made possible by its commitment, quality training methodologies, the services it offers, knowledge sharing with industry leaders, and professional approach.

Services

- Professional Web Design
- SEO and Internet Marketing
- Pay-Per-Click Campaign Management
- Link Building and E-commerce Solutions
- Web Application Development
- Multimedia Presentations
- Customized Software Development
- Business Process Outsourcing – IT / ITES

1.2 System Specification

1.2.1 Hardware Requirements

For the IoT Device (Field Unit):

- Microcontroller – ESP8266 (NodeMCU)
- Sensors – Soil Moisture Sensor (YL-69), Rain Sensor (YL-83)
- Actuation – 2-Channel Relay Module, DC Water Pump
- Alert System – Buzzer / LED Indicator
- Power Supply – 5V DC Adapter / USB Power
- Connectivity – Wi-Fi Network
- Additional Components – Jumper Wires, Breadboard / PCB

For the Monitoring System (Host PC / Smartphone):

- Processor – Intel Dual Core or above
- Processor Speed – 2.0 GHz or higher
- RAM – 4 GB (minimum 2 GB)
- Hard Disk – 250 GB or above
- Monitor – 15-inch colour monitor or higher
- Input Devices – Standard keyboard and mouse

- Smartphone – Android / iOS device with internet access (for Blynk App monitoring)

1.2.2 Software Specification

Front End	: Blynk App
Back End / Firmware	: NodeMCU ESP8266 programmable board
Programming Language	: Embedded C / Arduino
IoT Platform	: Blynk IoT Platform
Microcontroller IDE	: Arduino IDE
Operating System	: Windows 10 or above
Mobile Application	: Blynk App (Android / iOS)
Browser (optional)	: Google Chrome / Edge for the Blynk Web Dashboard

1.3 Software Description

1.3.1 Front End – Blynk App

Blynk is an IoT platform that lets a project be monitored and controlled from a mobile app or a web dashboard without the need to design a custom user interface. A datastream is created for every quantity that must travel between the device and the app – in this project, the soil-moisture reading, the rain status, and the pump control command each occupy their own virtual pin. The ESP8266 firmware pushes sensor values to these virtual pins over Wi-Fi, and Blynk instantly reflects the change as a gauge, an LED widget, or a toggle switch on the dashboard.

- Requires no native app development – widgets are configured visually in the Blynk console.
- Works over standard Wi-Fi, so the ESP8266 needs no additional radio hardware.
- Supports push notifications and e-mail alerts, used here to warn the farmer when the soil is dry.
- The same datastreams can be viewed from the mobile app or the web dashboard.

1.3.2 Back End – Embedded C on the Arduino IDE

The field-unit firmware is written in Embedded C and compiled for the ESP8266 using the Arduino IDE together with the ESP8266 board package and the Blynk library. This combination was chosen over a general-purpose scripting language because it produces a small, deterministic binary that starts reading sensors within milliseconds of power-up, which matters for a device that may run unattended in a field for months at a time.

- Direct register-level access to the analog-to-digital converter for fast, low-jitter soil-moisture sampling.

- A mature library ecosystem (ESP8266WiFi, Blynk, Ticker) that removes the need to hand-roll networking code.
- Low memory footprint, which leaves headroom for over-the-air update support in future revisions.
- Wide community support and documentation for the NodeMCU ESP8266 board.

2. SYSTEM STUDY

2.1 Existing System

In the existing irrigation system, farmers manually monitor soil conditions and control water supply. Irrigation is usually performed at fixed time intervals without considering real-time soil moisture or weather conditions. In some cases, basic timer-based systems are used, which activate the pump for a set duration regardless of the soil's actual requirement. These traditional methods often lead to over-irrigation or under-irrigation. Water wastage is common, especially during rainfall, when irrigation continues unnecessarily. Manual monitoring also increases labour costs and requires constant supervision. The lack of automation and environmental sensing makes existing systems inefficient and less reliable for modern agricultural needs.

2.1.1 Drawbacks

- High water wastage due to lack of monitoring.
- Overwatering or underwatering of crops.
- Requires continuous manual supervision.
- Increased labour costs.
- No rainfall detection mechanism.
- Reduced crop productivity due to inconsistent watering.

2.2 Proposed System

The proposed system introduces an automated irrigation mechanism using a soil moisture sensor and a rain sensor. The soil moisture sensor continuously monitors the moisture level in the soil. If the moisture drops below a predefined threshold, the microcontroller activates a relay to turn on the water pump. When sufficient moisture is reached, the pump is automatically turned off. The rain sensor detects rainfall and stops irrigation during rainy conditions. This system ensures optimal water usage and eliminates manual intervention. It improves efficiency, conserves water, reduces labour, and supports sustainable agricultural practices through smart and automated irrigation control.

2.2.1 Features

- Automatic soil moisture monitoring.
- Rain detection for smart irrigation control.
- Microcontroller-based automation.
- Water-saving mechanism.
- Low power consumption.

- Remote monitoring and control through the Blynk App.
- Easy installation and maintenance.

3. SYSTEM DESIGN AND DEVELOPMENT

3.1 File Design

The file design organizes the project into a compact, well-structured set of resources suited to a microcontroller-based deployment. The main firmware file (`irrigation_system.ino`) contains the setup and loop routines that initialise the sensors, connect to Wi-Fi, and drive the control logic. Project configuration – the Wi-Fi credentials, the Blynk authentication token, and the moisture thresholds – is kept in a separate header file (`config.h`) so that these values can be changed without touching the control logic itself.

On the cloud side, the Blynk platform maintains the datastream history for each virtual pin (soil-moisture value, rain status, pump state), which can be exported as a CSV log at any time from the Blynk web console. This removes the need for the field unit to manage its own database, keeping the firmware lightweight while still giving the farmer a complete watering history.

3.2 Input Design

The input design focuses on collecting environmental data reliably and letting the farmer adjust operating parameters without reflashing the firmware. The two physical inputs are the soil moisture sensor, which is read as an analog voltage on the ESP8266's ADC pin, and the rain sensor, which produces a digital HIGH/LOW signal depending on whether its plate is wet.

A third input path comes from the Blynk App itself: the dry-soil and wet-soil threshold values, and a manual override toggle that lets the farmer start or stop the pump on demand, are both exposed as writable datastreams. Every value received from the app is range-checked in firmware before it is applied, so an invalid entry from the dashboard cannot leave the pump in an unsafe state.

3.3 Output Design

The output design presents the current field status clearly, both at the device and remotely. Locally, a status LED and buzzer give an immediate indication of whether the pump is running. Remotely, the Blynk Web Dashboard displays a live soil-moisture gauge, a rain-status indicator, and a toggle showing whether the motor is on, mirroring exactly what is happening in the field.

When the soil moisture crosses the dry threshold, Blynk also sends a push notification and an e-mail alert to the registered account, so the farmer is informed even when the app is closed. All watering events are retained in the platform's datastream history, giving the farmer a simple way to review recent watering activity and adjust thresholds if the crop is being over- or under-watered.

3.4 Code Design

The firmware follows a modular, function-based structure so that each responsibility can be tested and maintained independently. Separate functions handle Wi-Fi connection management, sensor acquisition, threshold comparison, relay control, and communication with the Blynk cloud. The main loop is kept short: it reads the sensors, evaluates the irrigation decision, and lets the Blynk library service any pending communication, with a small delay between cycles to conserve power.

Error handling relies on simple guards rather than exceptions, which is standard practice in embedded C: a Wi-Fi reconnect routine runs automatically if the connection drops, and sensor readings are validated against expected ranges before being acted upon, so a loose wire or a momentary glitch cannot switch the pump on unnecessarily. This structured design keeps the system reliable, easy to extend, and straightforward to debug on hardware with limited memory.

3.5 System Development

3.5.1 Description of Modules

1. Soil Moisture Sensor Module

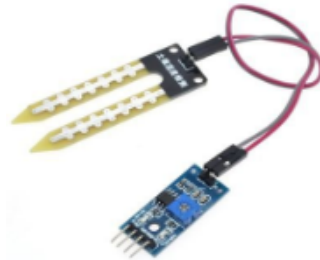


Fig 1: Soil Moisture Sensor (YL-69)

The Soil Moisture Sensor Module is responsible for measuring the water content present in the soil. It works on the principle of electrical conductivity, where the resistance between the sensor probes changes based on the moisture level. When the soil is dry, resistance is high; when the soil is wet, resistance decreases. The sensor continuously monitors the moisture condition and generates either analog or digital output signals. These signals are transmitted to the microcontroller for processing. By setting a threshold value, the system determines whether irrigation is required. This module ensures that crops receive water only when necessary, preventing over-irrigation and maintaining optimal soil conditions for healthy plant growth.

2. Rain Sensor Module



Fig 2: Rain Sensor Module (YL-83)

The Rain Sensor Module detects the presence of rainfall and prevents unnecessary irrigation during rainy conditions. It typically consists of a rain-sensitive plate that detects water droplets on its surface. When rainwater falls on the sensor, it changes the conductivity across the sensing plate, generating a signal. This signal is sent to the microcontroller, which immediately stops the irrigation process if the pump is running. The rain sensor helps conserve water by utilising natural rainfall effectively. It also protects crops from overwatering, which can damage roots and reduce productivity. This module

enhances the overall efficiency and intelligence of the irrigation system.

3. Microcontroller Module



Fig 3: NodeMCU ESP8266

The Microcontroller Module acts as the brain of the entire system. It receives input signals from both the soil moisture sensor and the rain sensor. Based on programmed logic and predefined threshold values, it decides whether the water pump should be turned ON or OFF. The microcontroller processes sensor data in real time and sends control signals to the relay module. It ensures accurate decision-making and smooth operation of the system. Additionally, it manages timing functions and power control, and it is responsible for maintaining the Wi-Fi connection used to publish sensor data to the Blynk cloud, making the irrigation process fully automated and remotely visible.

4. Relay Module

The Relay Module acts as an electrical switch that controls the operation of the water pump. Since the microcontroller operates at low voltage and current levels, it cannot directly drive the high-power pump. The relay acts as an intermediary component, allowing the microcontroller to safely control the pump. When the microcontroller sends a signal, the relay either completes or breaks the circuit connected to the pump. This ensures safe switching and protects the system from electrical damage.

5. Water Pump Module

The Water Pump Module is responsible for supplying water to the agricultural field. When activated by the relay, the pump draws water from a reservoir or water source and delivers it to the crops through irrigation pipes. The pump operates only when the soil moisture level falls below the required threshold and no rain is detected. This controlled operation ensures efficient water distribution and reduces unnecessary power consumption. The pump plays a vital role in maintaining consistent soil

moisture levels for crop growth.

6. Power Supply Module

The Power Supply Module provides the required electrical energy to all system components, including the sensors, microcontroller, relay, and pump. It converts the available power source (AC or battery) into regulated DC voltage suitable for the electronic components. A stable power supply ensures accurate sensor readings and reliable system performance. In advanced setups, this module can include voltage regulators, transformers, and even solar panels for renewable energy support. Proper power management enhances the durability and efficiency of the entire irrigation system.

4. TESTING AND IMPLEMENTATION

4.1 Testing

Testing is an important phase in the development of the IoT system using the ESP8266 connected to the Blynk platform. The testing process ensures that both hardware and software components function correctly and communicate reliably over the internet. Initially, individual components such as the ESP8266 module, sensors, and output devices (LED, buzzer, relay) were tested separately to verify proper connections and functionality. Sensor readings were checked to confirm accurate data acquisition.

Next, Wi-Fi connectivity testing was performed to ensure that the ESP8266 could successfully connect to a stable wireless network. The communication between the ESP8266 and the Blynk cloud server was verified by monitoring real-time data updates on the Blynk mobile application. Notification testing was also conducted to confirm that alerts were received instantly when triggered.

Integration testing was carried out to validate the complete system workflow, ensuring smooth interaction between hardware, cloud platform, and mobile interface. Finally, reliability and performance testing were performed by running the system continuously to check stability, response time, and error handling under different conditions.

The objectives of testing are as follows:

- To verify proper communication between the ESP8266 module and the Blynk cloud platform over Wi-Fi.
- To ensure accurate sensor data transmission and reliable real-time alerts through the Blynk mobile application.

Testing Issues

- The ESP8266 may face unstable or weak internet connection, causing delays or failure in sending data to the Blynk server.
- Insufficient or unstable power can lead to unexpected restarts or malfunction of the ESP8266 module.
- Incorrect wiring or improper calibration may result in inaccurate sensor readings or false triggers.
- Network latency or Blynk server downtime may cause delayed notifications or temporary data synchronization issues.

4.1.1 Testing Methodologies

Testing methodologies for the IoT system using the ESP8266 connected to the Blynk platform follow a systematic approach to ensure reliability, accuracy, and performance. The first methodology is Unit Testing, where individual components such as the ESP8266 module, sensors, relays, LEDs, and buzzer are tested independently to verify proper functionality. Each hardware component is checked for correct wiring and response.

The second methodology is Connectivity Testing, which ensures that the ESP8266 successfully connects to the Wi-Fi network and communicates with the Blynk cloud server. Data transmission between the device and the mobile application is monitored for consistency and speed.

The third methodology is Integration Testing, where all hardware and software components are combined and tested as a complete system. This verifies smooth interaction between sensors, microcontroller, cloud platform, and user interface.

Finally, Performance and Reliability Testing is conducted by running the system continuously under different network conditions to evaluate stability, response time, and error handling. These methodologies ensure the IoT system operates efficiently and reliably in real-world environments.

4.2 Types of Testing

System Testing

System testing is performed to evaluate the complete IoT system using the ESP8266 connected to the Blynk platform as a fully integrated unit. In this phase, all hardware components, such as the sensors, actuators, and the ESP8266 module, are connected and tested together with the software and cloud environment. The main objective is to ensure that the entire system works according to the project requirements.

During system testing, the ESP8266 is connected to a Wi-Fi network and linked to the Blynk application using the authentication token. Sensor inputs are triggered to verify whether the data is correctly processed and transmitted to the Blynk dashboard in real time. Output devices such as LEDs or buzzers are tested to confirm proper activation based on commands or sensor values.

Notification testing is also conducted to ensure alerts are received without delay. The system is observed under continuous operation to check stability, response time, and error handling. This testing ensures the IoT solution functions reliably in real-time environments.

Unit Testing

Unit testing is performed to verify the proper functioning of each individual component in the IoT system using the ESP8266 connected to the Blynk platform. In this phase, every hardware and software unit is tested separately before integrating them into the complete system. The ESP8266 module is first tested to ensure it powers on correctly and establishes a stable Wi-Fi connection. Basic

programs, such as LED blinking, are uploaded to confirm proper board configuration and communication with the Arduino IDE.

Next, individual sensors are tested to check whether they provide accurate readings and respond correctly to environmental changes. Output components such as LEDs, buzzers, or relays are also tested independently to ensure they activate when triggered. Additionally, the Blynk authentication token and virtual pins are verified to confirm proper data transmission between the ESP8266 and the Blynk mobile application.

By testing each module independently, errors can be identified and resolved early, ensuring smooth integration and reliable system performance in later stages.

Integration Testing

Integration testing is performed to verify the proper interaction between all hardware and software components in the IoT system using the ESP8266 connected to the Blynk platform. In this phase, individual modules such as the sensors, actuators, ESP8266 microcontroller, Wi-Fi network, and the Blynk mobile application are combined and tested as a complete system.

The testing process begins by connecting the sensors and output devices to the ESP8266 and uploading the integrated program. Once powered on, the ESP8266 is connected to a Wi-Fi network and linked to the Blynk cloud server using the authentication token. Sensor data is monitored in real time on the Blynk application to confirm successful communication between the hardware and cloud platform.

Alerts, notifications, and device control features are tested to ensure proper synchronization. Any delay in data transmission or mismatch in readings is identified and corrected. Integration testing ensures seamless coordination among components and confirms that the system operates reliably in real-world IoT environments.

Validation Testing

Validation testing is performed to ensure that the IoT system using the ESP8266 connected to the Blynk platform meets the intended project requirements and functions as expected under real-time conditions. This process verifies whether the complete system satisfies user needs and operational objectives.

Initially, the system is tested by triggering input devices such as sensors or switches to confirm that the ESP8266 correctly processes the input data. The output responses, such as LED indication, buzzer alerts, or relay activation, are observed to validate proper hardware functionality. Simultaneously, the Blynk mobile application is monitored to ensure that real-time data updates and notifications are received without delay.

Further validation includes testing the system under different network conditions to confirm stable Wi-Fi connectivity and reliable cloud communication. Multiple test cases are executed to verify accuracy, response time, and consistency. Continuous operation testing is also conducted to ensure long-term stability. Through validation testing, the system's performance, reliability, and compliance with user requirements are successfully confirmed.

System Implementation

System implementation testing is carried out after completing the development and integration of the IoT system using the ESP8266 with the Blynk platform. This phase ensures that the fully implemented system operates correctly in a real-time environment. Initially, the hardware components, such as the sensors, ESP8266 module, and output devices, are assembled and verified for proper physical connections. The firmware is then uploaded to the ESP8266 using the Arduino IDE.

Once deployed, the system is connected to a Wi-Fi network and linked to the Blynk cloud using the assigned authentication token. Real-time testing is performed by triggering the sensors and observing whether the corresponding data is accurately updated in the Blynk mobile application. Notifications and control commands are tested to confirm two-way communication between the device and the user interface.

The system is also monitored for stability by running it continuously to detect issues such as disconnections, delays, or hardware failures. This testing ensures the implemented IoT solution functions reliably under practical conditions.

5. CONCLUSION AND FUTURE ENHANCEMENT

5.1 Conclusion

The Automatic Irrigation System Using Rain Sensor and Soil Moisture Sensor provides an efficient and reliable solution for modern agriculture. By automating the irrigation process, it reduces water wastage and minimizes manual effort. The integration of soil moisture and rain sensors ensures that crops receive the right amount of water at the right time. This improves crop health, increases productivity, and supports sustainable farming practices. The system is cost-effective, energy-efficient, and easy to implement in small and large agricultural fields. Overall, the project demonstrates how smart technology can enhance agricultural efficiency and contribute to water conservation.

5.2 Future Enhancement

In the future, this system can be enhanced by adding temperature and humidity sensing (for example, a DHT22 module) alongside the existing soil moisture and rain sensors, so watering decisions can also account for evaporation rate, not just soil dryness. Weather forecasting data can be integrated to make predictive irrigation decisions ahead of expected rainfall. Solar power can be used to make the system energy independent for remote fields. The system can also be expanded with data analytics on the historical Blynk datastream to optimize watering schedules over a full growing season. These enhancements will transform the irrigation system into a fully smart and intelligent agricultural solution.

BIBLIOGRAPHY

Reference Books

- Banzi, M., & Shiloh, M. (2014). *Getting Started with Arduino*. Maker Media.
- Monk, S. (2016). *Programming Arduino: Getting Started with Sketches*. McGraw-Hill Education.
- Blum, J. (2019). *Exploring Arduino: Tools and Techniques for Engineering Wizardry*. Wiley.
- Kolban, N. (2018). *Kolban's Book on ESP8266*. Leanpub.
- Richardson, M., & Wallace, S. (2012). *Getting Started with Raspberry Pi*. O'Reilly Media.
- McEwen, A., & Cassimally, H. (2014). *Designing the Internet of Things*. Wiley.
- Bahga, A., & Madiseti, V. (2015). *Internet of Things: A Hands-On Approach*. Universities Press.
- Minoli, D. (2017). *Building the Internet of Things with IPv6 and MIPv6*. Wiley.
- Reese, G. (2018). *Cloud Application Architectures*. O'Reilly Media.
- Monk, S. (2018). *Arduino IoT Cloud for Developers*. Packt Publishing.
- Wilmshurst, T. (2010). *Designing Embedded Systems with PIC Microcontrollers*. Newnes.

Website References

- <https://blynk.io>
- <https://www.espressif.com/en/products/socs/esp8266>
- <https://www.arduino.cc>
- <https://randomnerdtutorials.com>
- <https://www.iotforall.com>

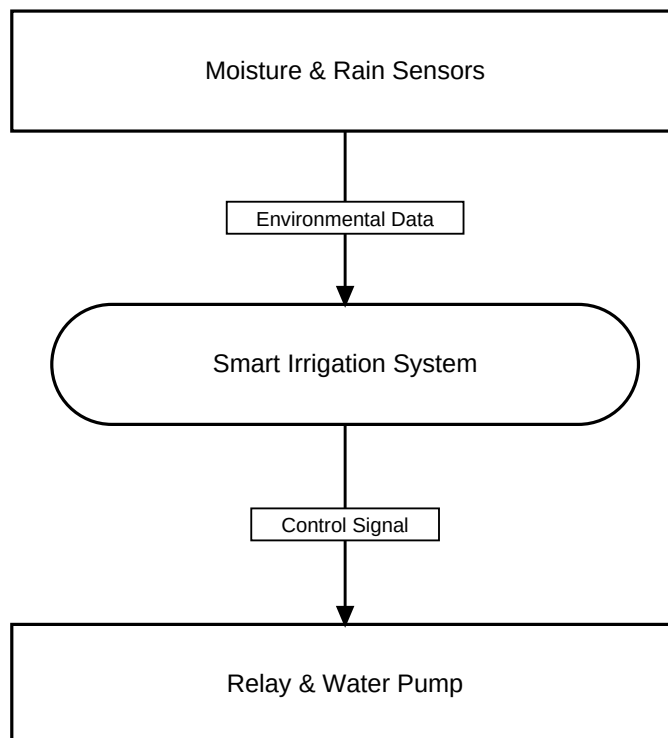
APPENDIX

Symbols of Data Flow Diagram

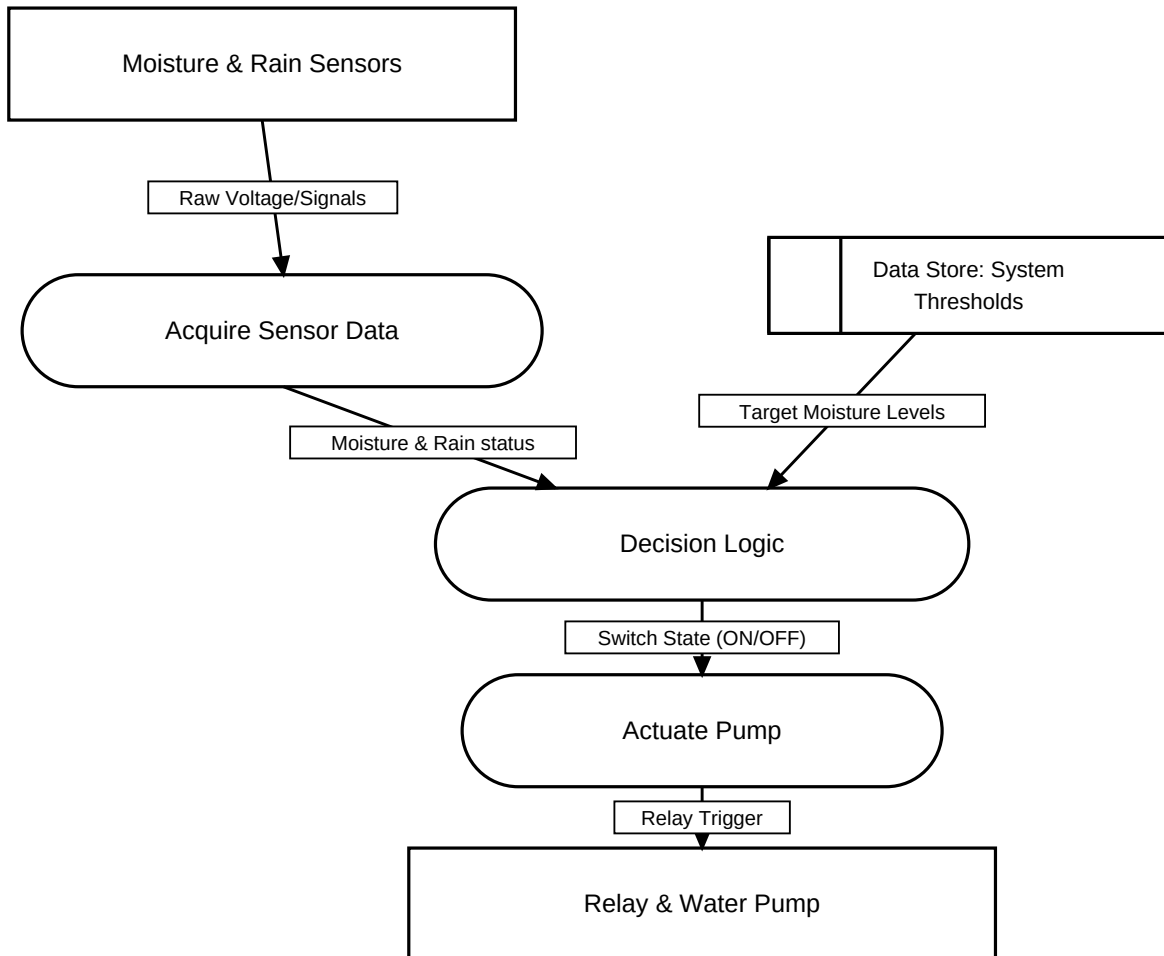
Rectangle Shape	Entity
Rounded Rectangle	Process
Arrow Head	Data Flow
Open Rectangle (double-lined)	Data Store
Curved-edge Box	Reports

A. Data Flow Diagram

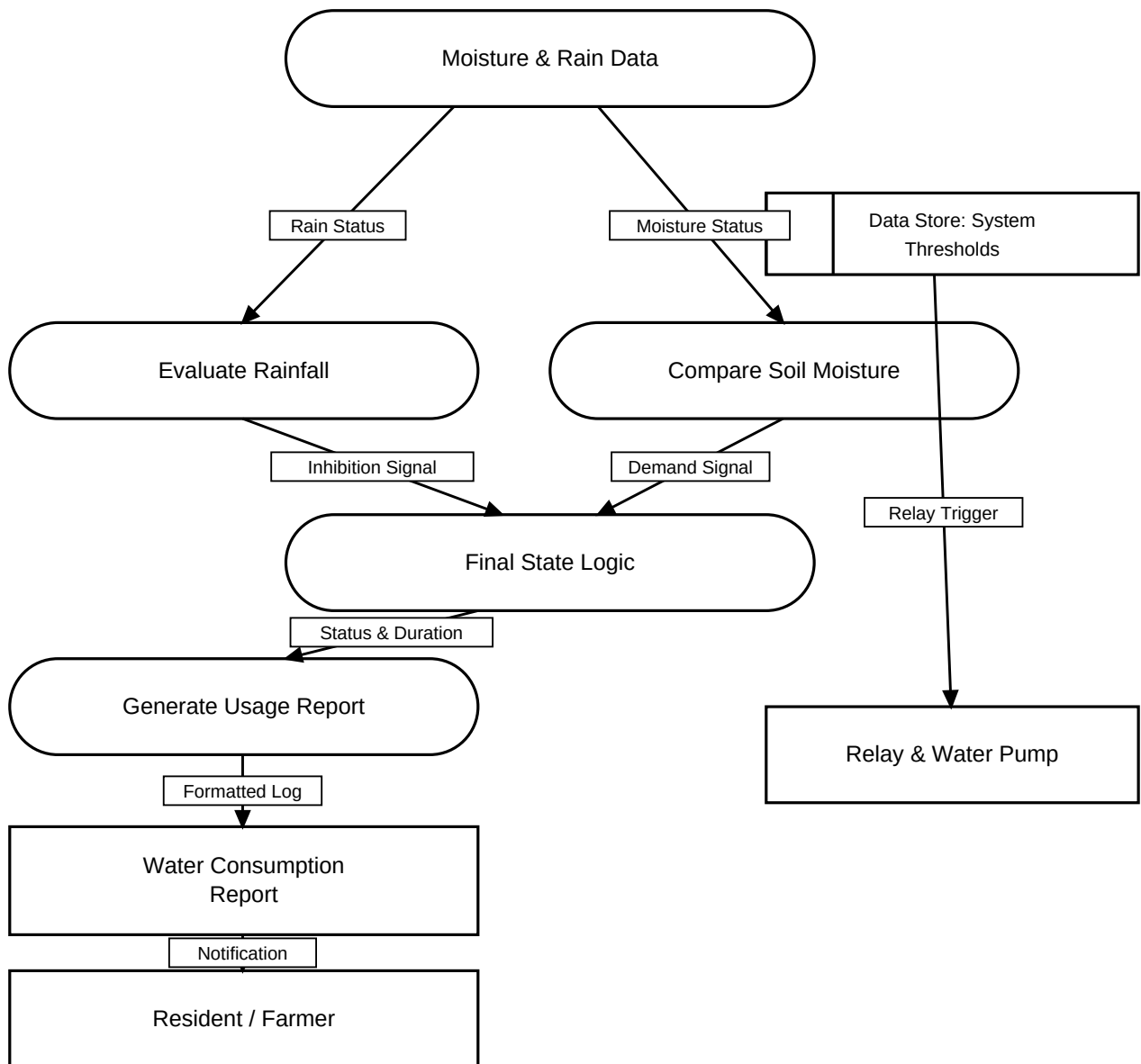
Level: 0



Level: 1



Level: 2



B. Working Diagram

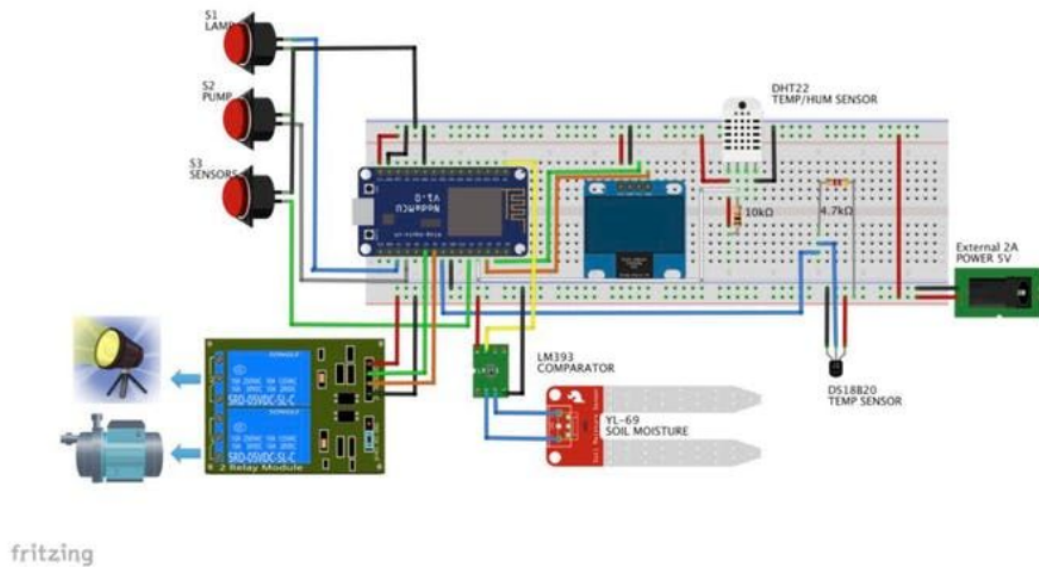


Fig 4: Working diagram of the Automatic Irrigation System showing the interconnection of sensors, the NodeMCU controller, the relay module, and the water pump for automatic irrigation control. The DHT22 and DS18B20 sensors shown are provisioned on the board for the temperature/humidity future-enhancement described in Section 5.2.

C. Sample Source Code

The following is the core firmware sketch, written in Embedded C for the Arduino IDE using the ESP8266 board package and the Blynk library, matching the software specification in Section 1.2.2. Virtual pin V0 carries the soil-moisture reading, V1 the rain status, and V2 the pump/motor control toggle – the same three datastreams shown on the Blynk Web Dashboard in Fig 5.

```
// =====
//  AUTOMATIC IRRIGATION SYSTEM - FIELD UNIT FIRMWARE
//  Board   : NodeMCU ESP8266
//  Cloud   : Blynk IoT Platform
//  IDE     : Arduino IDE (Embedded C / C++)
//  Version : 1.0.0
//  =====

#define BLYNK_TEMPLATE_ID   "TMPLxxxxxx"
#define BLYNK_TEMPLATE_NAME "Smart Irrigation and Monitoring"
#define BLYNK_AUTH_TOKEN    "YOUR_BLYNK_AUTH_TOKEN"

#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>

// ----- Wi-Fi credentials -----
char ssid[] = "YOUR_WIFI_NAME";
char pass[] = "YOUR_WIFI_PASSWORD";

// ----- Hardware pins -----
const int SOIL_SENSOR_PIN = A0;  // Analog input - YL-69 soil moisture
const int RAIN_SENSOR_PIN = D5;  // Digital input - YL-83 rain sensor
const int PUMP_RELAY_PIN  = D6;  // Digital output - relay driving the pump
const int STATUS_LED_PIN  = D4;  // Onboard status LED

// ----- Thresholds -----
int soilDryThreshold = 600;  // above this -> soil is dry, start watering
int soilWetThreshold = 350;  // below this -> soil is wet, stop watering
bool manualOverride   = false; // set from the Blynk app "Turn on Motor" widget

BlynkTimer timer;

// Read the moisture sensor and publish it to the dashboard (virtual pin V0)
int readSoilMoisture() {
  int value = analogRead(SOIL_SENSOR_PIN);
  Blynk.virtualWrite(V0, value);
  return value;
}

// Read the rain sensor and publish rain status to the dashboard (virtual pin V1)
bool isRaining() {
  bool raining = (digitalRead(RAIN_SENSOR_PIN) == LOW);  // LOW = wet plate
  Blynk.virtualWrite(V1, raining ? 1 : 0);
  return raining;
}
```

```
}

void pumpOn(const char *reason) {
  digitalWrite(PUMP_RELAY_PIN, HIGH);
  digitalWrite(STATUS_LED_PIN, HIGH);
  Blynk.logEvent("watering_event", reason);
}

void pumpOff() {
  digitalWrite(PUMP_RELAY_PIN, LOW);
  digitalWrite(STATUS_LED_PIN, LOW);
}

// Core decision logic, called on a timer every few seconds
void evaluateIrrigation() {
  int soil    = readSoilMoisture();
  bool raining = isRaining();

  if (manualOverride) {
    pumpOn("Manual override from app");
    return;
  }

  if (raining) {
    pumpOff();           // never water while it is raining
    return;
  }

  if (soil > soilDryThreshold) {
    pumpOn("Soil moisture below threshold");
  } else if (soil < soilWetThreshold) {
    pumpOff();
  }
  // between the two thresholds: leave the pump in its current state (hysteresis)
}

// Blynk app "Turn on Motor" toggle -> virtual pin V2
BLYNK_WRITE(V2) {
  manualOverride = (param.asInt() == 1);
}

void setup() {
  pinMode(PUMP_RELAY_PIN, OUTPUT);
  pinMode(STATUS_LED_PIN, OUTPUT);
  pinMode(RAIN_SENSOR_PIN, INPUT);
  pumpOff();

  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
  timer.setInterval(5000L, evaluateIrrigation); // evaluate every 5 seconds
}
```

```
void loop() {  
  Blynk.run();  
  timer.run();  
}
```

D. Sample Screens

Web Dashboard

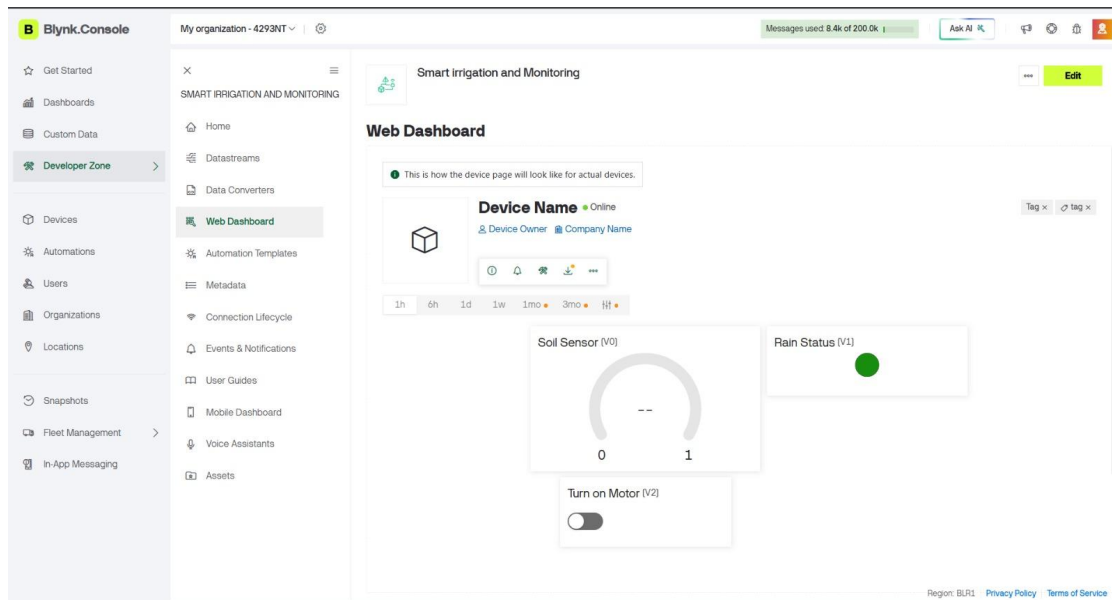


Fig 5: Blynk Web Dashboard showing the Soil Sensor gauge (V0), Rain Status indicator (V1), and the Turn-on-Motor toggle (V2).

Form Description: This screen shows the Blynk dashboard, which displays the live soil moisture level and rain status, and lets the farmer switch the pump on or off from the app.

Received Mail

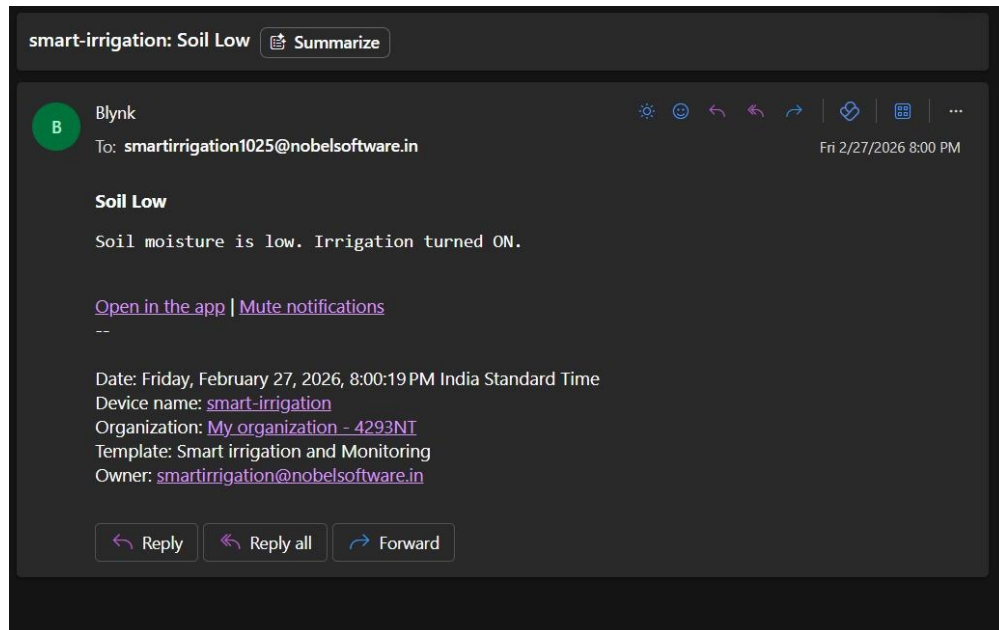


Fig 6: Automated e-mail alert sent by Blynk when the soil moisture drops below the configured threshold and irrigation is turned on.

Form Description: This screen shows the e-mail notification received from the Blynk platform, confirming that low soil moisture was detected and irrigation was started automatically.